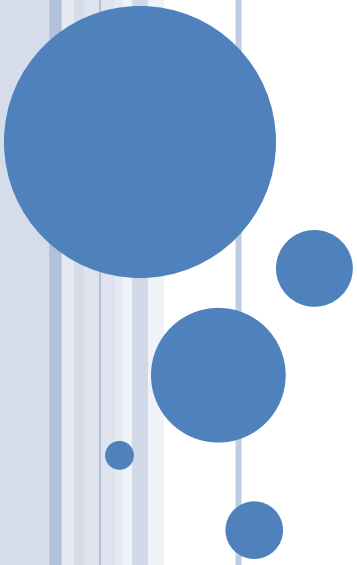
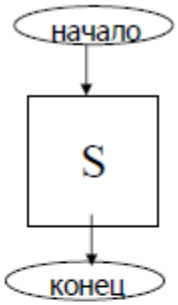


Теория автоматов и формальных языков

Метод Флойда верификации программ

Является ли программа правильной?



Пусть мы имеем исходный код из 100 000 строк С кода.
Вопрос: "*Эта программа правильна?*"

- Правильность программы можно разделить на
  синтаксическую правильность и
  семантическую правильность (корректность)
  - *Синтаксическая правильность* –правильность использования языка программирования при написании программы
  - *Семантическая правильность, или корректность* – это правильность по отношению к конкретной спецификации – требованиям к тому, что программа должна делать. Ответ на вопрос о корректности программы нельзя ожидать, пока нам не сказано, что программа должна делать
- Без информации о том, какую задачу программа должна решать, можно выполнить только "статический анализ кода" - найти неиспользуемый код, утечки памяти или ресурсов, явные ошибки именования-разыменования и т.п., но вопрос о корректности останется открытым

Программа может быть корректной либо некорректной только по отношению к заданной спецификации

НУЖНО ИМЕТЬ описание того, что программа должна делать

Верификация и тестирование

Два метода выявления ошибок в программах: тестирование и верификация

ТЕСТИРОВАНИЕ: проверка правильности работы программы на конкретных значениях входных параметров, на конкретных сценариях ее работы – сравнение того, что мы требуем от программы, с тем, что она выдает при прохождении тестового примера

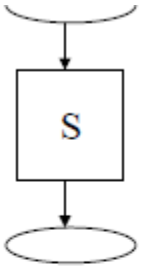
ВЕРИФИКАЦИЯ (доказательство корректности): формальное доказательство выполнения требований к работе программы. Это доказательство проводится на модели программы, при всех возможных значениях входных параметров, на всех возможных траекториях ее работы

Тестирование не может обеспечить приемлемый уровень надежности современных программных комплексов, особенно если они состоят из нескольких параллельно работающих подсистем

Верификация, доказательство корректности – одно из важнейших направлений исследований в области Computer Science с 1960-х годов



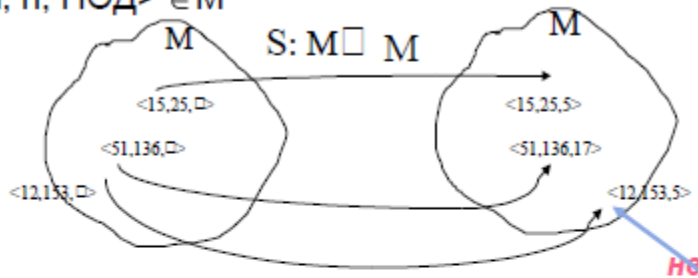
Тестирование программ преобразования данных (трансформационных, императивных программ)



S вычисляет НОД от m и n

M – множество всех троек натуральных чисел

<m, n, НОД> ∈ M

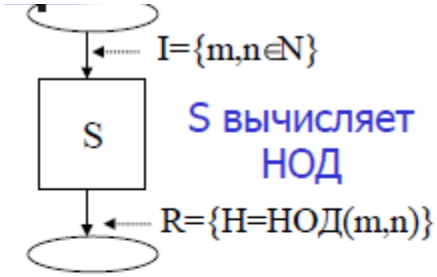


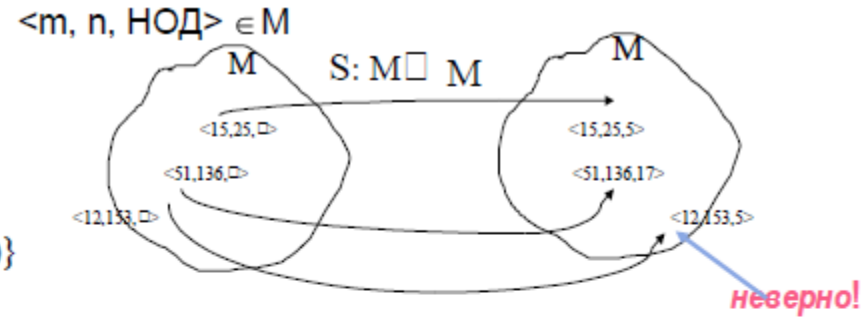
Программа преобразования данных – частичная функция из множества состояний памяти в себя

Состояния памяти – это наборы значений переменных программы, значений регистров и точки (Счетчика команд) в программе (все то, что запоминается при прерывании)

Тестирование – вычисление этой функции на отдельных конкретных наборах исходных данных (при некоторых начальных состояниях)

Тестирование и верификация программ обработки данных (императивных программ). Наибольший общий делитель (НОД)





Полное тестирование программы нахождения НОД требует $2^{64}=10^{19}$ тестов

"*Тестированием нельзя доказать правильность программы*"- Эдсгер Дейкстра

ВЕРИФИКАЦИЯ - проверка правильности работы программы на ***ВСЕХ возможных*** входных наборах

Сформулируем УТВЕРЖДЕНИЕ для программы вычисления НОД:
Если m и n – натуральные числа в начале программы, ***то*** при завершении программы S переменная H будет равна НОД m и n.

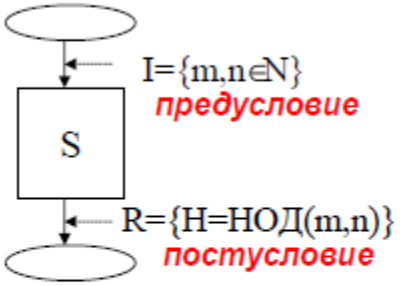
Это утверждение – предикат, который может быть либо истинным, либо ложным для заданной программы S. Этот предикат интерпретируется на множестве всех возможных программ

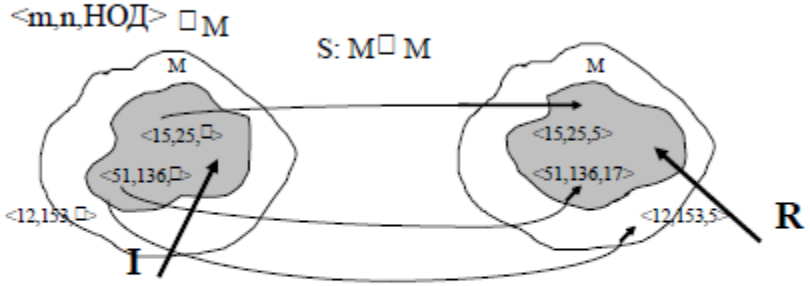
Спецификация программ обработки данных

Программу будем рассматривать, как преобразователь на множестве всех ее возможных ***состояний***

Состояние – набор значений переменных программы

Число состояний бесконечно





Как ***конечным образом*** описать все исходные и "правильные" финальные состояния???

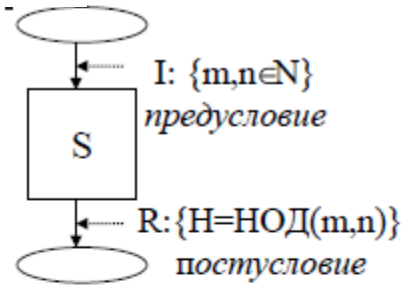
Средством описания ***подмножеств*** (бесконечных) множества является **предикат.**

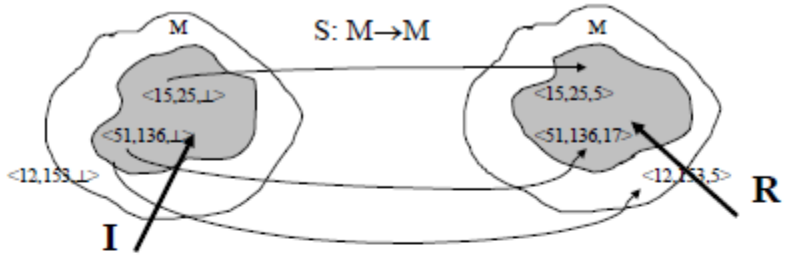
Например: x>y – все те пары x и y, в которых x > y (бесконечное число пар)

z+2x = y - определяет все тройки <x, y, z>, в которых это отношение соблюдается

Состояние <51, 136,17> - удовлетворяет предикату H=НОД(m, n), а <12,153,5> - нет

Свойство спецификации программ





Спецификация требований не связана с алгоритмом!

Примеры:

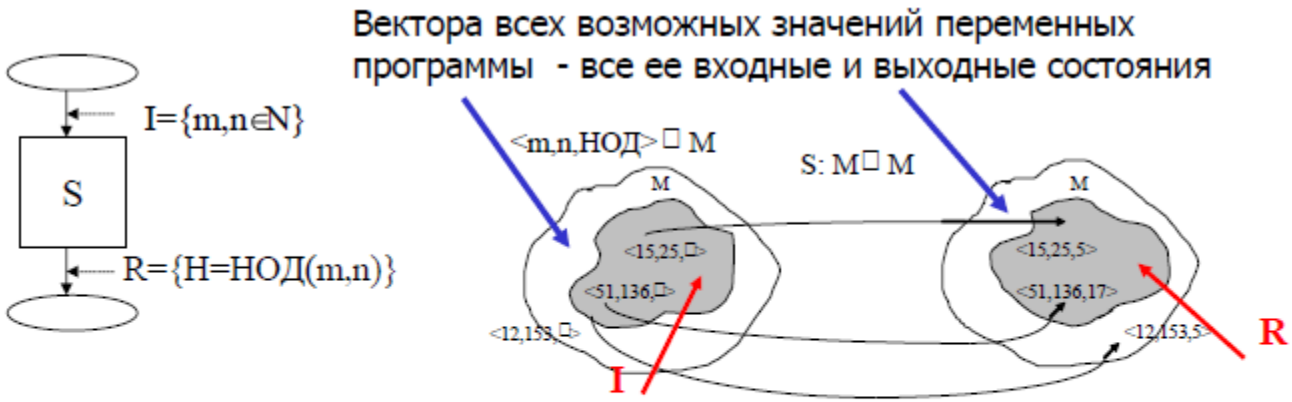
все сортировки I:: (∀i: 0≤i≤n) A[i]∈N R:: (∀i: 0≤i≤n-1) A[i]≤A[i+1]

спецификация всех возможных программ нахождения НОД

I:: m∈N ∧ n∈N R:: H = НОД(m, n)

Предикат определяет множество (обычно бесконечное) состояний программы, на которых этот предикат истинен

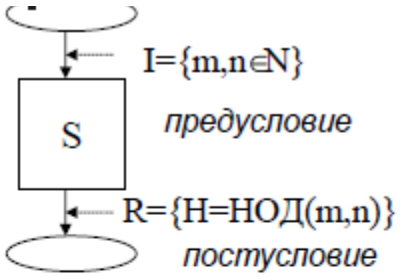
Программа вычисления НОД двух чисел

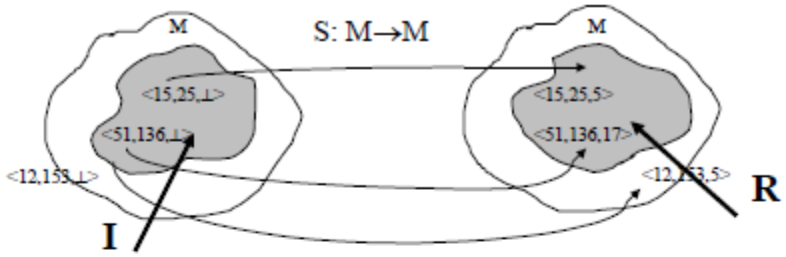


I – предусловие: предикат, который задает все такие значения входных переменных, для которых мы гарантируем правильную работу программы. Например, для программы извлечения квадратного корня x≥0

R – постусловие: задает соотношение между входом и выходом программы, т.е. правильный результат после завершения работы программы

Частичная и полная корректность программ





Формальная спецификация программы в двух формах:

A: *"**Если** предусловие* **I** *истинно до начала работы программы* **S**
И **S** *завершится,*
ТО после завершения **S** *постусловие* **R** *будет истинным"*

Это частичная (partial) корректность программы: **{ I } S { R }**

B: *"**Если** предусловие* **I** *истинно до начала работы программы* **S**,
ТО **S** *завершится (**завершение программы гарантируется**)*
И после завершения **S** *постусловие* **R** *будет истинно"*

Это полная (тотальная, total) корректность программы: **[I] S [R]**

Total correctness = Partial correctness + Termination

Примеры спецификаций

Для циклической незавершающейся программы утверждение:

```
{True}
 begin
      while True do x:=x
 end
 {True}
```

истинно (поскольку программа никогда не завершится)

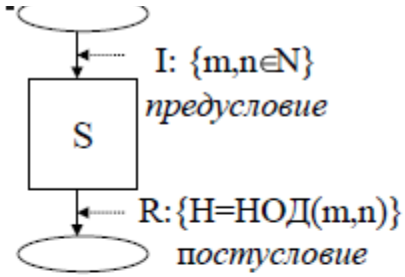
Для этой же программы утверждение:

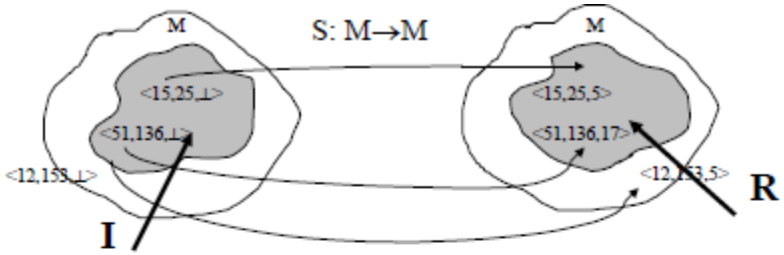
```
[True]
 begin
      while True do x:=x
 end
[True]
```

ложно (поскольку программа никогда не завершится)



Как проверить, что программа удовлетворяет спецификации?





Пусть заданы:

программа S,
предусловие **I** и
постусловие **R**

Как проверить истинность утверждения {I} S {R} –
"если I истинно при входе в программу S, и программа завершается, то после завершения S утверждение R станет истинным"?

Тестирование, верификация, валидация

Процессы и методы, направленные на повышение качества ПО

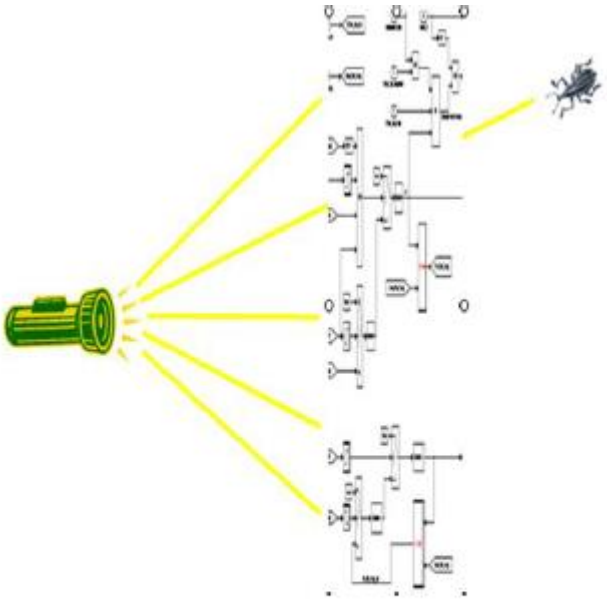
- ***Валидация*** – набор приемов и методов подтверждения того, что разработано то, что требует заказчик
- ***Верификация*** – набор формальных приемов и методов подтверждения того, что разрабатываемая система удовлетворяет формальным установленным требованиям (формальной спецификации)
- ***Тестирование*** – управление *выполнением* готовой программы с целью обнаружения несоответствия ее поведения установленным требованиям (процесс выполнения программы с намерением найти ошибку, а не подтвердить правильность программы)

Обычно тестирование ПО – проверка работы готовой системы на наборах тестовых данных в фиксированных сценариях. Это ДИНАМИЧЕСКИЙ анализ, проверка свойств при моделировании прогонов программы

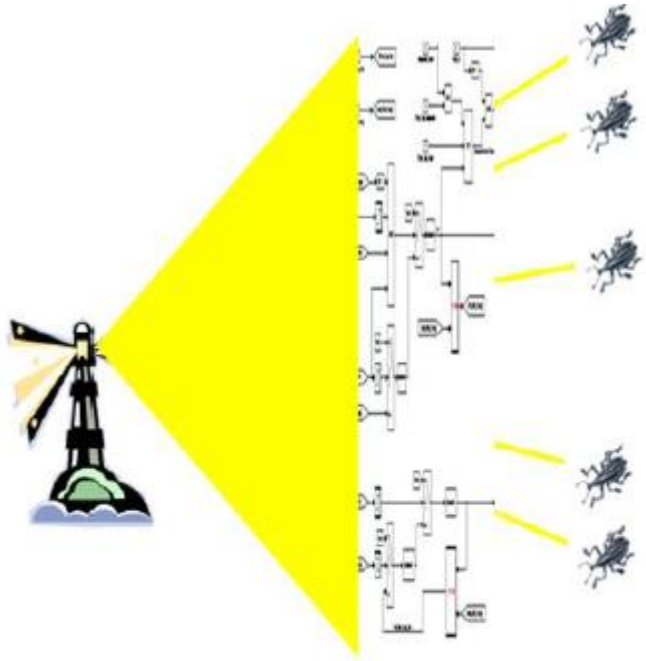
Формальная верификация – статический анализ, формальное ДОКАЗАТЕЛЬСТВО правильности текста программы: проверка выполнения формально определенных требований на формальной модели системы



Тестирование vs верификация (из презентации фирмы Rockwell-Collins)

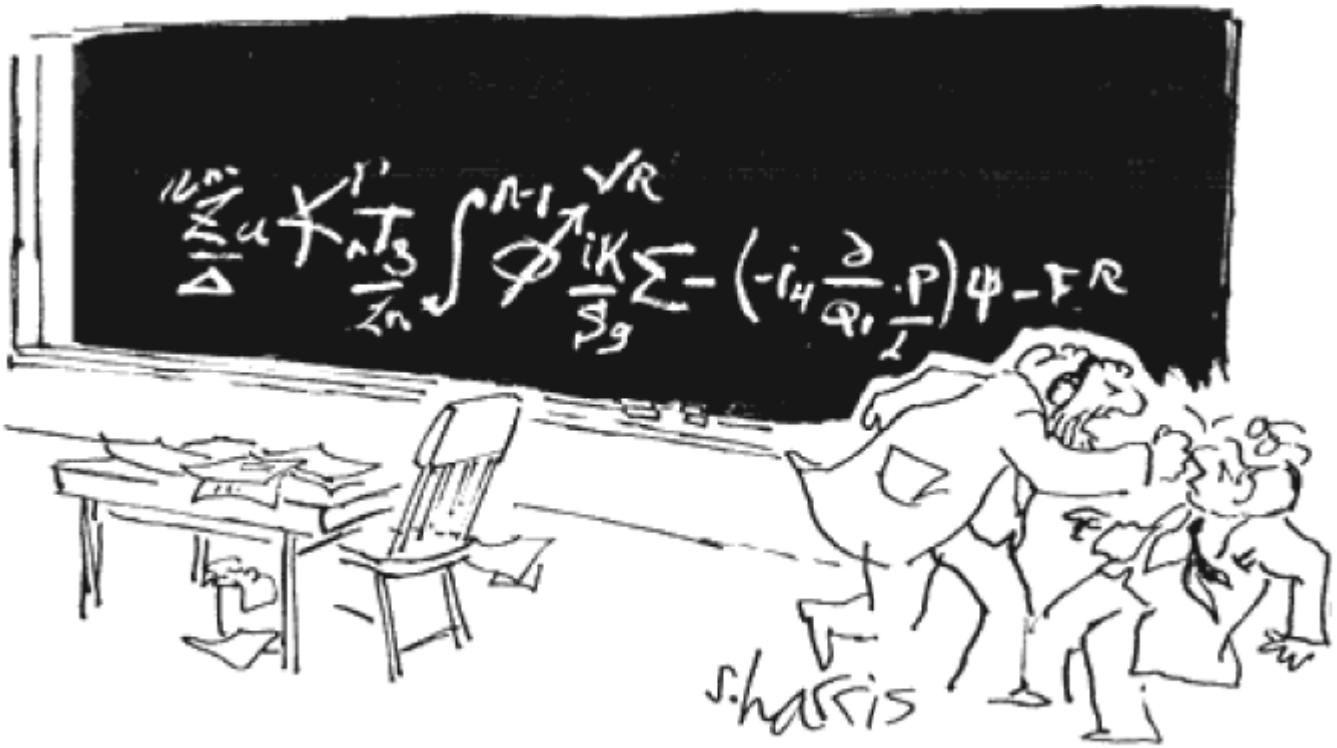


Тестирование: процесс выполнения программы с целью найти ошибку



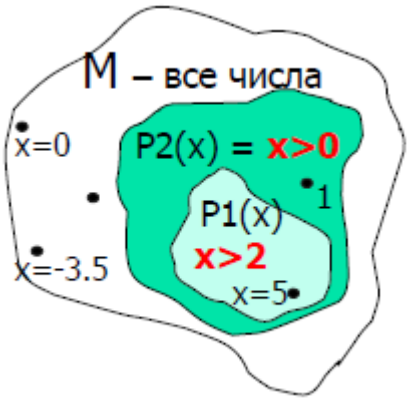
Верификация: процесс подтверждения того, что программа удовлетворяет формальным требованиям при всех возможных вычислениях

Доказать корректность программы непросто!



"You want proof? I'll give you proof!" Sidney Harris

Предикат P1 более сильный, чем P2



Пусть P1(x) и P2(x) – два предиката на множестве состояний программы

Определение. Предикат P1(x) более сильный, чем предикат *P2(x)* (а *P2(x)* слабее, чем *P1(x)*), если выполнение условия *P1(x)* влечет выполнение условия *P2(x)*

формально $P1(x) \Rightarrow P2(x)$

Например, условие P1(x) ={x > 2} сильнее условия P2(x)={x > 0}: при выполнении P1 выполняется и P2

Пример: P1(x) определяет множество квадратов,
P2(x) – множество прямоугольников

True – самое слабое условие,
все элементы M удовлетворяют True,

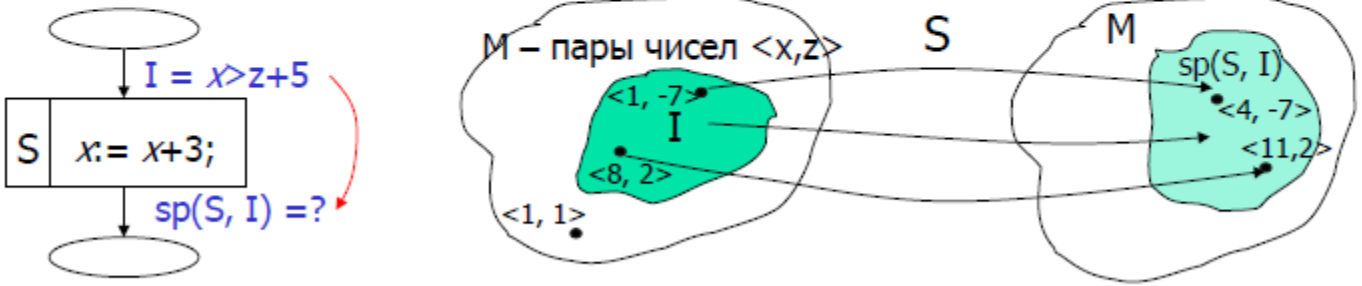
False – самое сильное условие,
ни один элемент M не удовлетворяет False

Программа как преобразователь предикатов

Программа преобразует данные, следовательно изменяется ее состояние.
Изменяются и соотношения между программыми переменными

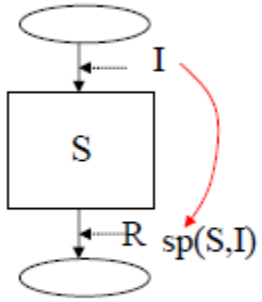
Программу можно рассматривать как *ПРЕОБРАЗОВАТЕЛЬ ПРЕДИКАТОВ* (Р. Флойд). По предусловию I в начале программы S можно вычислить ***сильнейшее постусловие*** (strongest postcondition), sp(S, I) – зависит от S и от I

Определение. Сильнейшее постусловие программы sp(S,I) - это предикат, определяющий все те состояния программы S, в которые преобразуются по завершении S все состояния, которые удовлетворяли предикату I до старта S



После завершения программы S значение *x* возрастет на 3.
Поэтому, если I = *x* > z+5 до выполнения S=x:=x+3, то sp(S, I) = *x* > z+8

Корректность программы при вычисленном sp(S, I)



Пусть сильнейшее постусловие sp построено

У нас есть программа и три предиката:

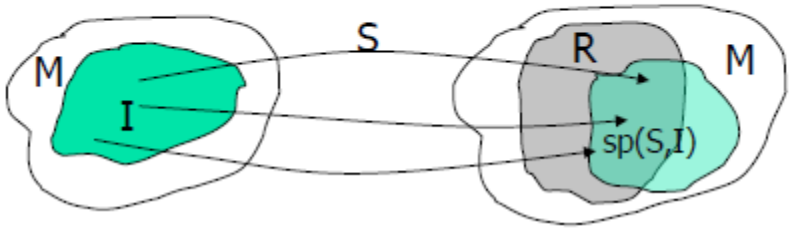
S - программа

I – предусловие - условие, которому должны удовлетворять программные переменные до выполнения программы

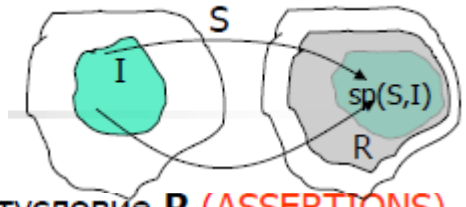
R – постусловие, которому, мы считаем, должны удовлетворять значения переменных программы после выполнения программы

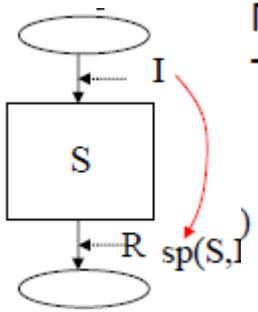
sp(S,I) – сильнейшее постусловие, которому удовлетворяют программные переменные после завершения программы S, если они удовлетворяли предусловию I

Какое между ними должно быть соотношение, чтобы программа S была корректна?



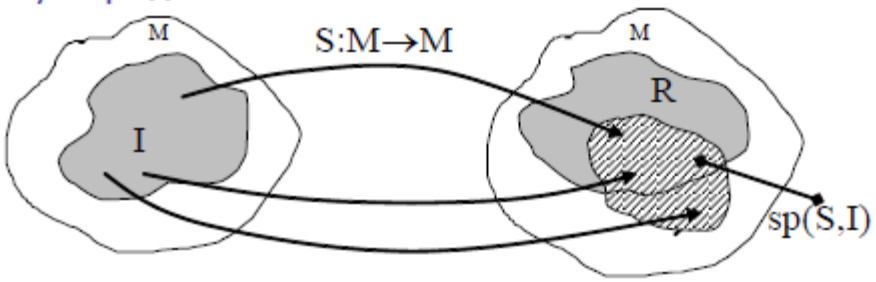
Триада Хоара: {I} S {R}





Пусть заданы S, предусловие **I** и постусловие **R** (ASSERTIONS)

Тройка (триада) Хоара {I} S {R} – *это утверждение*
если I *истинно до начала* S, ***то*** *после завершения* S
утверждение R станет *истинным*



Программа S (частично) корректна тогда и только тогда, когда sp(S,I) целиком лежит в R, т.е. когда формула sp(S,I)⇒R общезначима

Теорема. $\models\{I\}S\{R\} \Leftrightarrow sp(S, I) \Rightarrow R$

Верификация программы –задача логического вывода

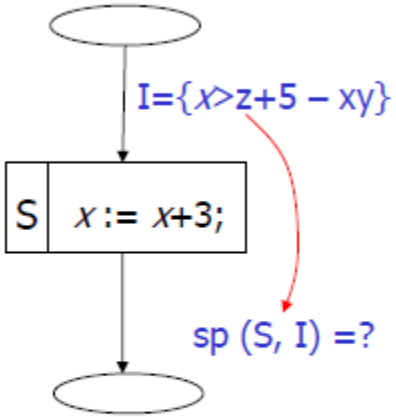
КАК НАЙТИ strongest postcondition - sp(S,R)

для блоков программ?



Сильнейшее постусловие для оператора присваивания

Как *формально* определить правила преобразования *любого* предиката операторами языка?



Возьмем простейший оператор – оператор присваивания

Чему равен предикат

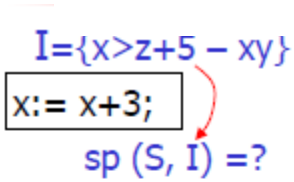
sp (x := x+3, x>z+5 – xy) =?

strongest postcondition

программа

precondition

Как оператор присваивания преобразует предикат?



Определим *формально*, как оператор присваивания преобразует *любой* предикат. Пусть I(x) – предусловие оператора x :=f(x)

Обозначим x′ *новое* значение *x после завершения* программы

x′= f(x) : новое x равно f(старое x) (*именно равно, не присвоить!*)

Тогда sp(S, I) = I(x) (предусловие НЕ изменилось)

(старое x не изменилось! Утверждение I сформулировано для старых значений x, и это соотношение для *старого* значения x не изменилось)

Но после завершения S предикат sp должен быть выражен через новое x, т.е. через x′. Как выразить старое значение x через новое x′ ?

Связь старого x и нового x′: $\boldsymbol{x = f^{-1}(x')}$ - *обратная функция*

программа предусловие

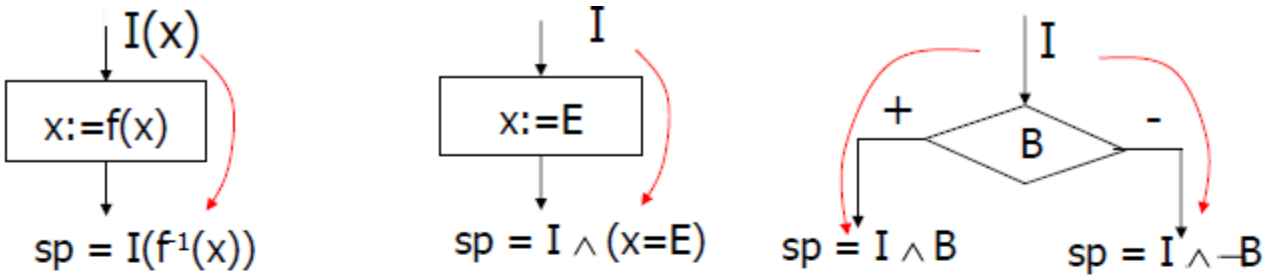
$$sp\ (\ x:=f(x),\quad I(x)\ ) = I\ (\ f^{-1}(\ x\ )\ )$$

формальная семантика оператора присваивания

Для нашего примера: S: x′ = x+3; x = x′-3; $f^{-1}(x) = x - 3$;

Поэтому sp(S, I) = (x-3) > z+5 – (x-3)y ≡ x > z+8 – xy + 3y

Формальная семантика операторов языка блок-схем



Для верификации программы с каждым ее оператором и с их комбинациями связываются преобразования предикатов, что является формальным выражением их свойств (их смысла, их семантики)

Формальная семантика операторов программы

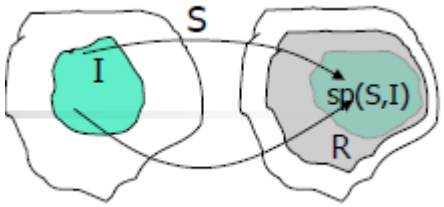
ЕСЛИ для любого предусловия I сможем вывести сильнейшее постусловие sp(S, I), ТО понятно, как работает оператор S (известна формальная семантика S)

Такая семантика строится только для ограниченного множества операторов:

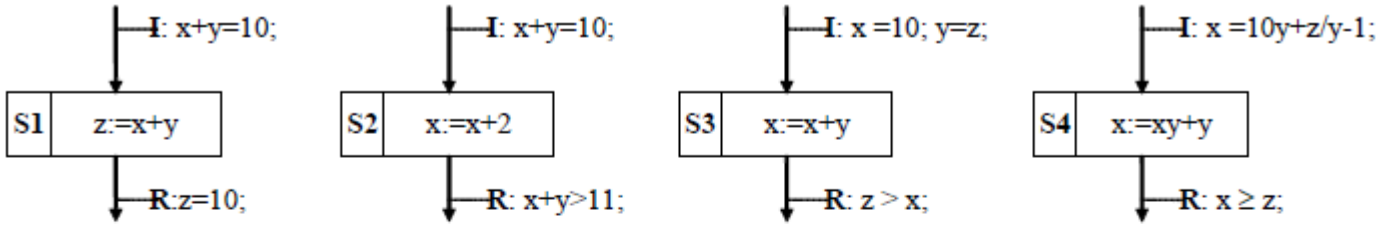
- на структурном языке программирования (фактически, блок-схемы)
- без использования адресной арифметики
- без подпрограмм
- без взаимодействия с окружением

… … …

Проверка корректности простейших программ



S частично корректна относительно предусловия I и постусловия R iff sp(S, I)⇒R



Доказательство каждой программы сводится к доказательству теоремы

$sp(S, I) \Rightarrow (?)\ R$

S1: sp(S1, I): $(x+y=10)\ \&\ (z=x+y)$; $(x+y=10)\ \&\ (z=x+y) \Rightarrow (?)\ z=10$ **ДА!** S1 корректна

S2: sp(S2, I): $(x-2+y=10)$; $(x-2+y=10) \Rightarrow (?)\ x+y>11$ **ДА!** S2 корректна

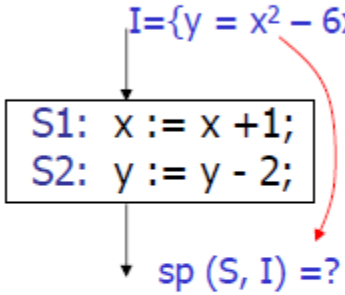
S3: sp(S3, I): $(x-y=10)\ \&\ (y=z)$; $(x-y=10)\ \&\ (y=z) \Rightarrow (?)\ z>x$ **НЕТ!** S3 некорректна

S4: sp(S4, I): $(x-y)/y = 10y+z/y-1 \equiv x = 10y^2+z$; $x = 10y^2+z \Rightarrow (?)\ x \geq z$ **ДА!** S4 корректна

Школьный пример

"Если параболу $y = x^2 - 6x + 2$ сдвинуть на одну единицу вправо и на две единицы вниз, то график какой функции получится?"

Задачу можно сформулировать в наших терминах:



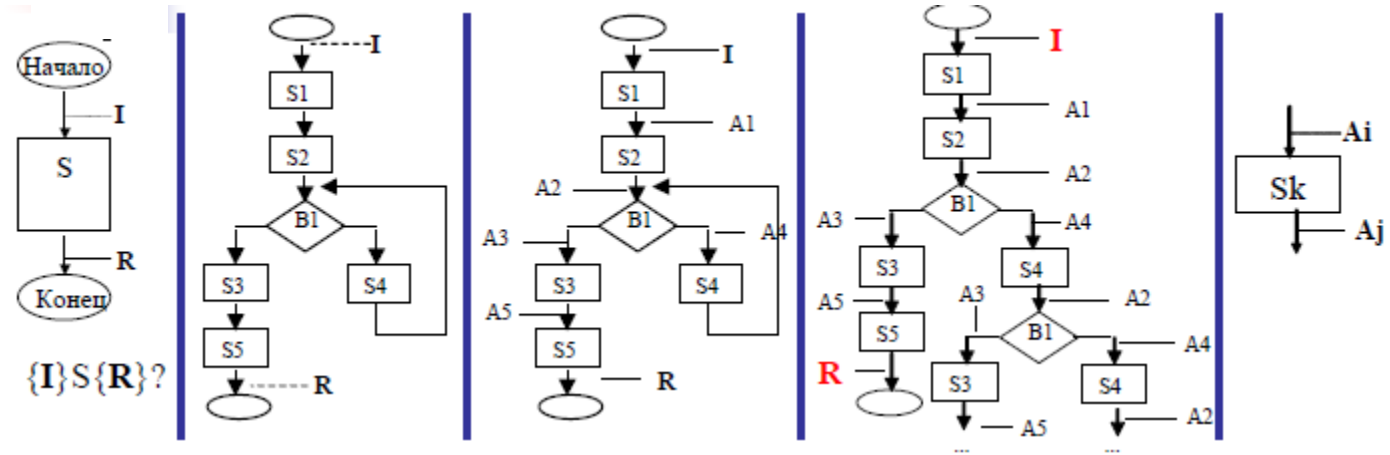
$$sp(S_1; S_2, I) = sp(S_2, sp(S_1, I))$$

Сначала предикат "протаскиваем" через оператор S1, затем – через оператор S2

$$sp(S, I) = sp([\ x:=x+1;\ y:=y-2\ ], \{\ y = x^2 - 6x + 2\ \}) \equiv$$
$$sp([\ y:=y-2], \{\ sp([\ x:=x+1], \{\ y = x^2 - 6x + 2\ \})\ \}) \equiv$$
$$sp([\ y:=y-2], \{\ y = x^2 - 6x + 2\ \}_{x \leftarrow x-1}\ \}) \equiv$$
$$sp([\ y:=y-2], \{\ y= (x-1)^2 - 6(x-1) + 2\ \}) \equiv$$
$$\{\ y = x^2 - 8x + 9\ \})\ \}_{y \leftarrow y+2} \equiv$$
$$\{\ (y+2) = x^2 - 8x + 9\ \})\ \} \equiv$$
$$\{\ y = x^2 - 8x + 7\ \}$$

Ответ: после сдвига получим график параболы $y = x^2 - 8x + 7$

Индуктивный метод Роберта Флойда доказательства частичной корректности программ (1967)

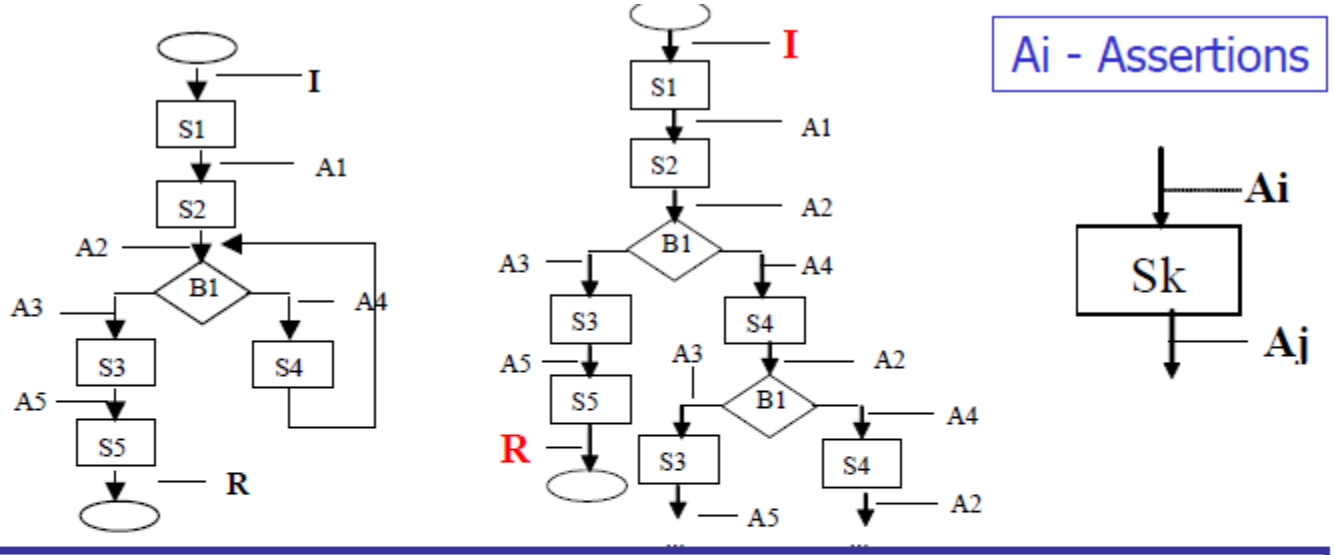


Роберт Флойд: каждая стрелка блок-схемы снабжается утверждением, и для каждого блока Si доказывается истинность утверждения $\{A_i\}\ S_k\ \{A_j\}$

Теорема. *Пусть для каждой тройки $\{A_i\}\ S_k\ \{A_j\}$ доказана ее истинность. Тогда, если **I** истинно, то каждое утверждение A_r, встретившееся на пути вычислений (в том числе и R), истинно.* ***Доказывается по индукции***

Следствие. ***Если программа завершается*** (т.е. если R когда-нибудь встретится на вычислении) то, *если **I** истинно на входе в S и каждая тройка $\{A_i\}\ S_k\ \{A_j\}$ истинна, то* истинно утверждение, представленное тройкой { ***I*** } S { ***R*** }

Если для некоторых Ai, Sk и Aj доказать {Ai} Sk {Aj} нельзя



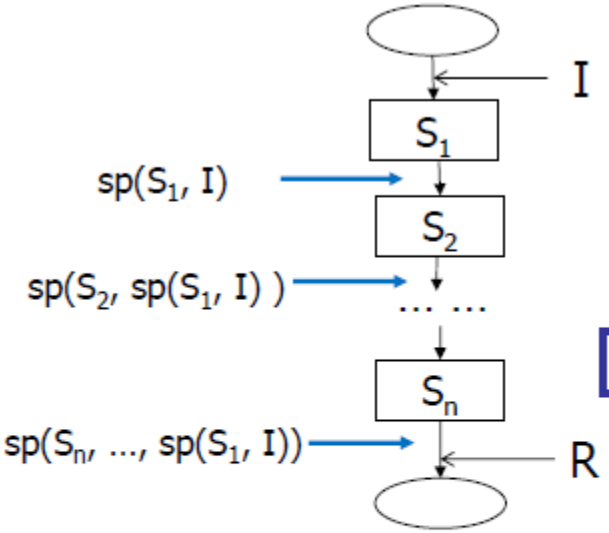
Если при сформулированных ассертах невозможно доказать корректность какого-либо блока Sk, то ИЛИ программа неверна, ИЛИ неверно построены утверждения на стрелках программы

Если доказана корректность ВСЕХ утверждений $\{A_i\}\ S_k\ \{A_j\}$, то программа может не завершаться: гарантии того, что ВСЕ вычисления завершатся, нет

КАК проверять реальные программы?



Доказательство корректности ациклических программ



Пусть программа –
последовательность операторов
$S_1; S_2; S_3; \ldots, S_n$
и заданы предусловие I, постусловие R

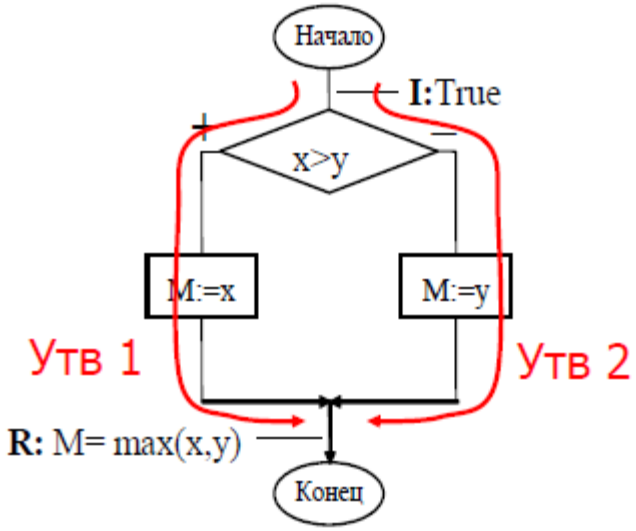
Выполняем "протаскивание" предикатов

Дополнительных утверждений
придумывать не нужно!

Для доказательства корректности программы, состоящей из последовательности операторов, нужно доказать теорему:

$$sp(S_n, sp(S_{n-1}, \ldots, sp(S_1, I) \ldots)) \Rightarrow R\ ?$$

Доказательство корректности ациклических программ. “Протаскивание” предикатов



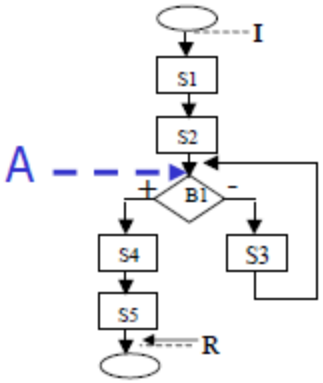
Для доказательства аннотированной программы поиска максимума двух значений, x и y, доказываем две теоремы:

sp(S,I)⇒R: Утверждение 1: $True \wedge (x>y) \wedge (M=x) \Rightarrow M=max(x,y)$

Утверждение 2: $True \wedge \neg(x>y) \wedge (M=y) \Rightarrow M=max(x,y)$

Но это – определение максимума двух чисел!

Инварианты циклов: понимание поведения программы



Для доказательства корректности нет необходимости придумывать все утверждения, после каждого блока

Достаточно придумать только инварианты – по одному такому утверждению в каждом цикле, которые остаются истинными при каждом прохождении цикла

Если инвариант цикла A придуман, то доказательство циклической программы сводится к доказательству трех теорем:

T1: $sp([s1; s2], I) \Rightarrow A$;

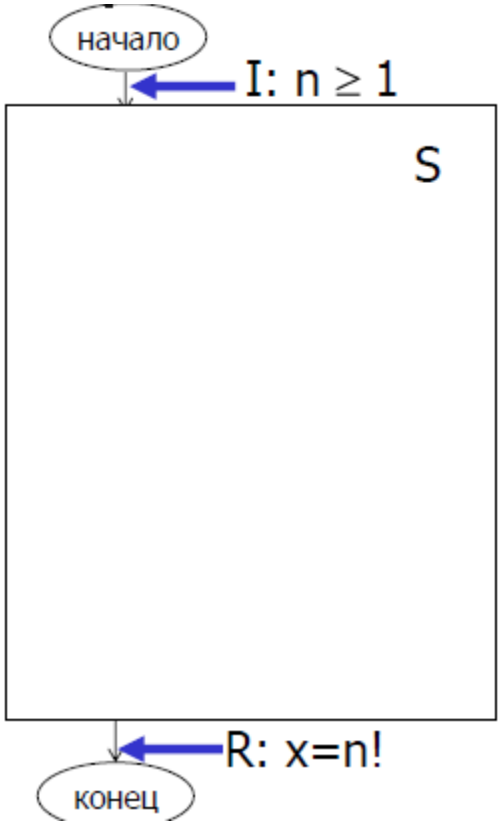
T2: $sp([B1-; s3], A) \Rightarrow A$;

T3: $sp([B1+; s4; s5], A) \Rightarrow R$;

Чтобы придумать правильный инвариант, нужно вникнуть в замысел разработчика, понять суть алгоритма

Инвариант цикла обычно отражает идею, лежащую в основе алгоритма

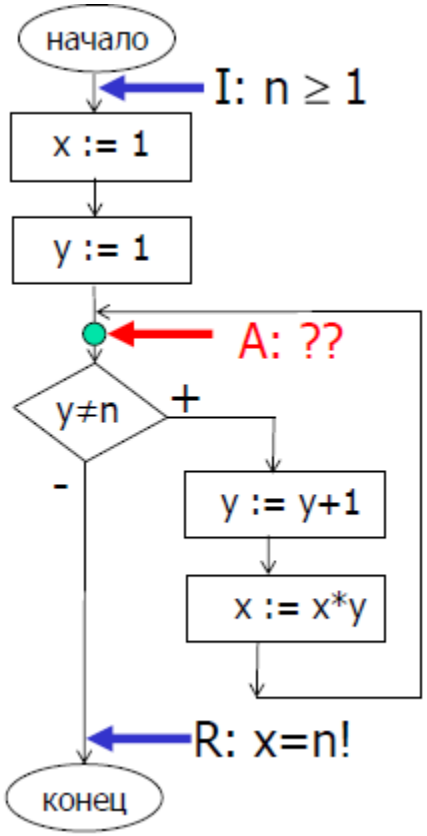
Пример: программа n! – инвариант цикла



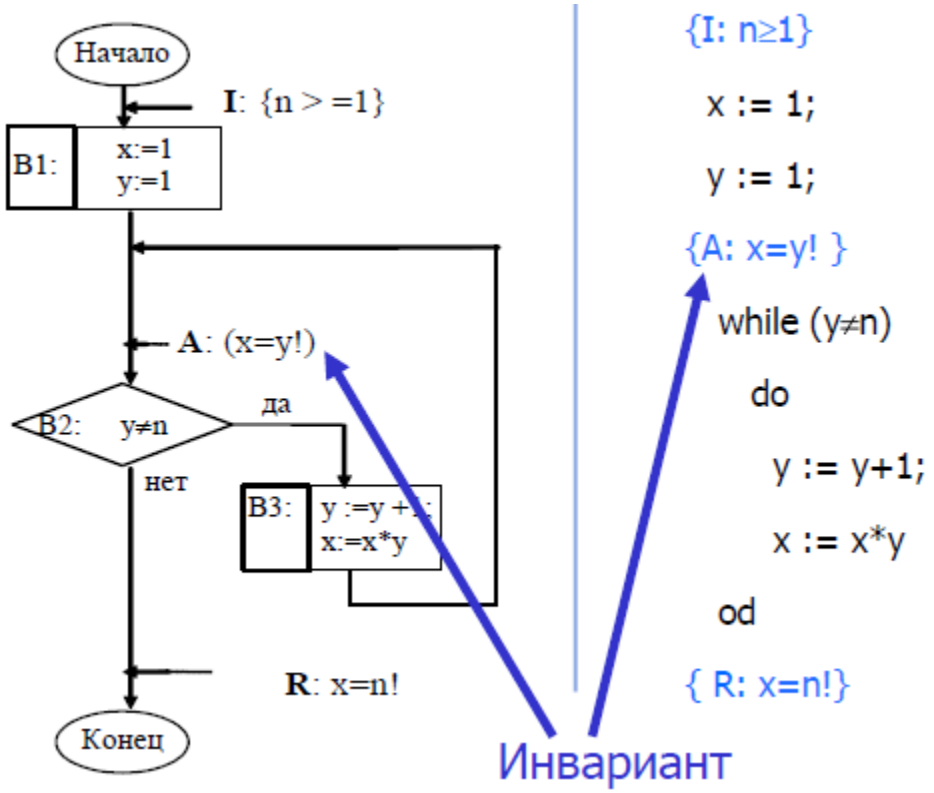
Как выбрать инвариант цикла???

A: x=y!

Каждый раз, когда попадаем в точку, значение x равно факториалу y



Пример аннотированной программы: n!




```
{I: n≥1}
  x := 1;
  y := 1;
{A: x=y! }
  while (y≠n)
     do
       y := y+1;
       x := x*y
  od
{ R: x=n!}
```

Инвариант

Инвариант цикла: задача о кофейных зернах

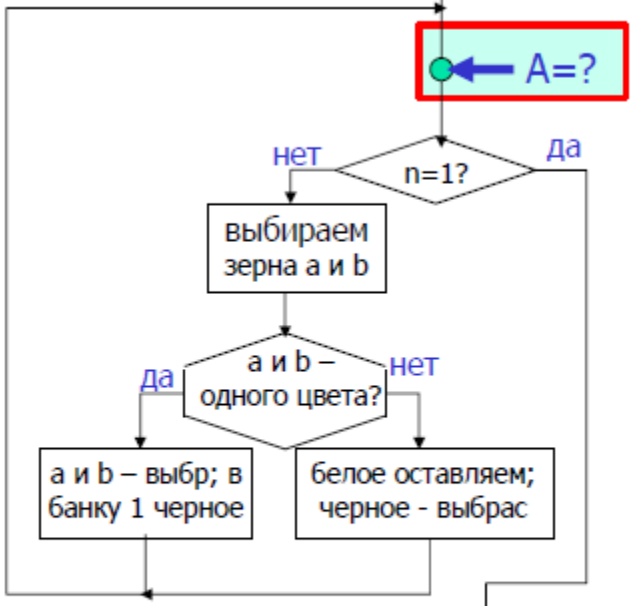


В банке несколько черных и белых кофейных зерен. Повторяем, пока можно:

Выбираем из банки случайно два зерна;
- если они одного цвета, то их выбрасываем, а в банку кладем черное;
- если они разного цвета, то черное выбрасываем, а белое возвращаем

В конце концов, останется одно зерно.

ВОПРОС: какого цвета оставшееся зерно, если известно, сколько было вначале черных и сколько белых зерен?



Тесты обычно не помогают.
Нужно алгоритм ПОНЯТЬ
Для этого нужно построить ИНВАРИАНТ

Инвариант А (то, что сохраняется на каждом шаге): белые выбрасываются только парами, т.е. четность белых не меняется.

Решение : *было нечетное число белых – останется белое*
было четное число белых – останется черное

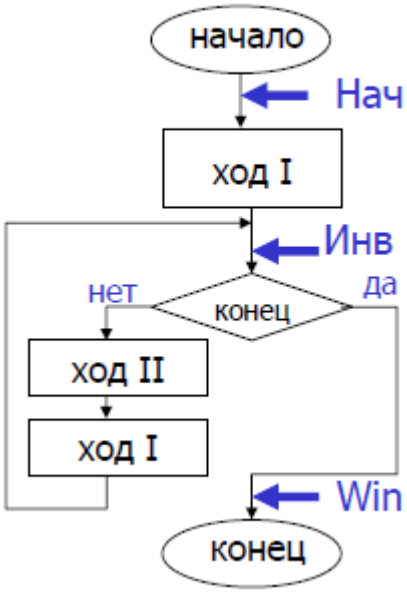
Инвариант = выигрышная стратегия в игре

Дано N=376 спичек. Играющие по очереди берут произвольное число, от 1 до 4х спичек. Выигрывает игрок, взявший последнюю спичку.



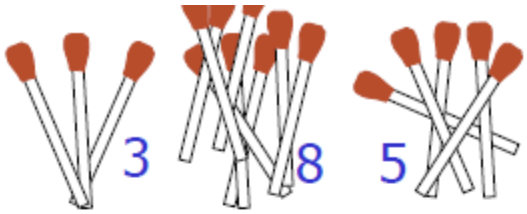
376 штук

- Ищем выигрышную стратегию для 1 игрока
- Каково частное значение n, по которому игра заканчивается? Ясно, что n=0. Это частный случай инварианта, приводящего I игрока к выигрышу.
- Какое общее условие на оставшееся число предметов должно оставаться истинным, если II игрок может выбрать любое число от 1 до 4х, а потом I игрок – произвольное в том же диапазоне?
- Ясно, что если II возьмет любое от 1 до 4х, то I потом может выбрать тоже от 1 до 4-х, так, чтобы сумма двух выборок была равна 5. Т.е. пара: ход II, потом ход I может на каждом шаге текущее значение числа спичек уменьшить на 5.
- Оставляем N такое: при делении на 5 остаток 0!
- Стратегия для выигрыша I – надо своим ходом всегда оставить N=0(mod5)

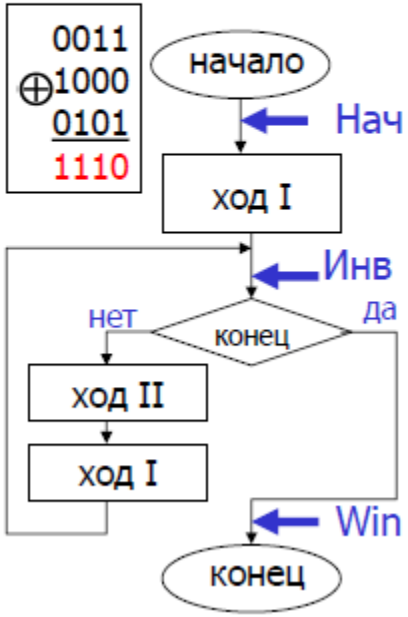


Игра Ним: инвариант ==> выигрышная стратегия

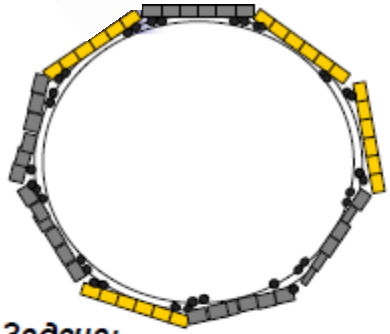
Дано несколько кучек спичек. Играющие по очереди берут произвольное число, но не менее 1 спички из одной какой-либо кучки.
Выигрывает игрок, взявший последнюю спичку.



- Какая стратегия выбора хода? Ключ – поддержание игроком инварианта выигрыша. Как его построить?
- Выигрыш – когда остается 0 спичек. Какой инвариант построить, чтобы 0 спичек было частным случаем этого утверждения, и после любого хода противника 1-й игрок мог его восстановить?
- ИНВАРИАНТ: НИМ_Сумма (поразрядная сумма по модулю 2 (XOR) числа спичек в всех кучках) равна 0.
  Если НИМ_Сумма ≠ 0, ее за один ход можно сделать нулевой. Если она равна 0, любой ход ее делает ненулевой
- Например, у нас НАЧ = 3, 8 и 5, т.е. 0011, 1000 и 0101. Поразрядная сумма (XOR) ≠ 0. Выигрышная стратегия у первого. Он должен своим ходом инвариант восстановить. Может взять 2 спички из 2-й, чтобы осталось 6. Тогда НИМ_сумма станет 0: 0011⊕0110⊕0101.
- Своим ходом I всегда может привести систему в состояние, на котором этот инвариант выполняется. Выигрышная стратегия I такова: после любого хода игрока II инвариант нужно восстанновить



Инвариант: задача о кольце вагонов



Задача:

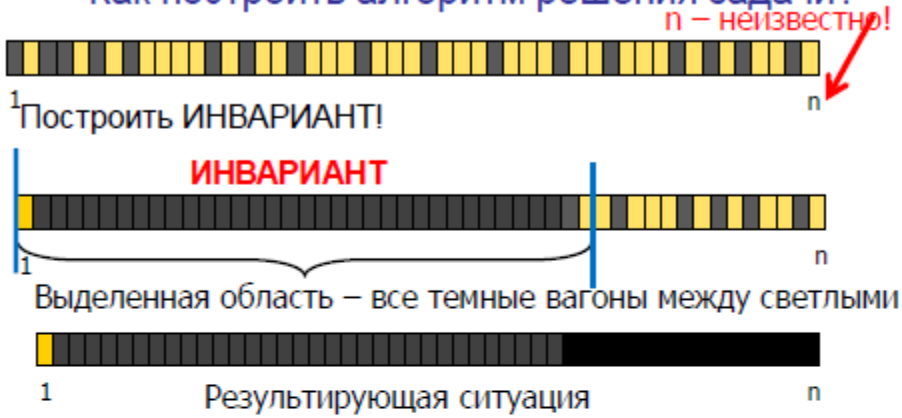
Некоторое число вагонов сцеплены между собой по кругу. Внутри ходит машинист, который может только включать или выключать свет в вагонах.

Как машинисту подсчитать количество вагонов?

Вначале в вагонах свет горит случайным образом. Никаких пометок в вагонах проставлять нельзя.

(Кружок ФТШ 566 С.Пб)

Как построить алгоритм решения задачи?



Понятие инварианта дает понимание того, как построить алгоритм решения и как определить неизвестное число n

1. Создаем начальный инвариант (пустое множество темных вагонов между двумя светлыми)
2. На каждом шаге увеличиваем ширину инварианта, ***пока левая его граница не сольется с правой***

Какое решение? Как поддерживать инвариант?? Как определить, что пришли к конечной ситуации?

Доказательство корректности программы вычисления факториала

x = n!

Программа находит значение факториала n с помощью умножений



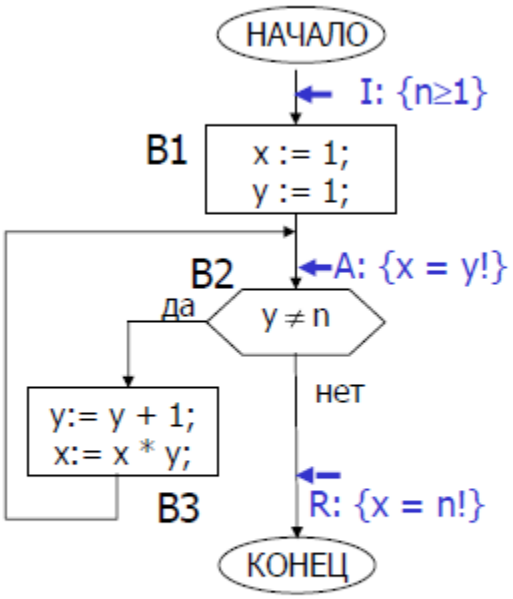
Доказательство корректности программы требует доказательства трех теорем

T1: sp(B1, I) ⇒A

T2: sp([B2да; B3], A) ⇒A

T3: sp(B2нет, A) ⇒R

Доказательство частичной корректности программы вычисления факториала



Доказательство теорем

T1: sp(B1, I) ⇒(?) A

sp(B1, I) = (n≥1 ∧ x=1 ∧ y=1)

(n≥1 ∧ x=1 ∧ y=1) ⇒(?) (x = y!) ДА!

T2: sp([B2да; B3], A) ⇒(?) A

sp([B2да; B3], A) = sp(B3, sp (B2да; A)) =
sp(B3, x = y! ∧ y≠n)=
(x/y = (y-1)! ∧ (y-1) ≠ n)

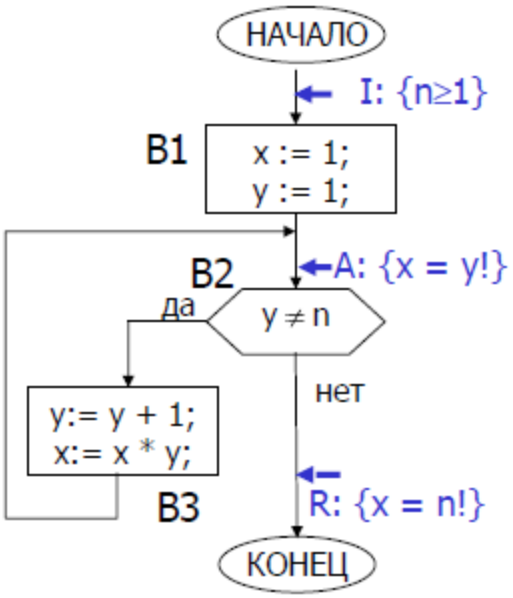
(x/y = (y-1)! ∧ (y-1) ≠ n) ⇒(?) (x = y!) ДА!

T3: sp(B2нет, A) ⇒(?) R

sp(B2нет, A) = (x = y! ∧ ¬y≠n)

(x = y! ∧ y = n) ⇒(?) (x = n!) ДА!

Доказательство частичной корректности программы вычисления факториала



Почему частичная корректность?

Все доказательства справедливы, если число n – вещественное, например, 4.5

Вначале n=4.5, x = 1, y=1,
затем

y=2, x=2;

y=3, x=6;

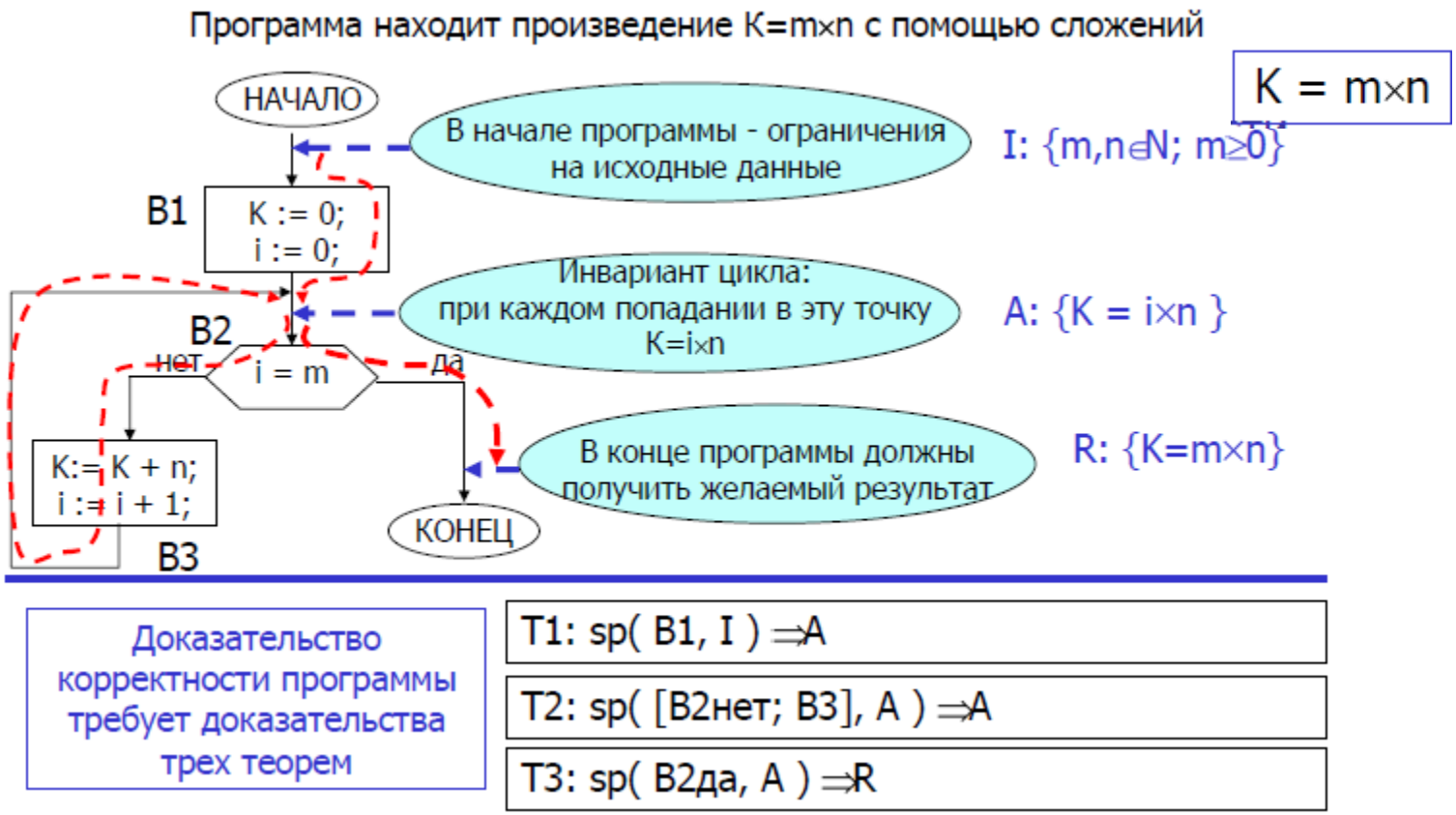
y=4, x=24;

y=5, x=120;

...

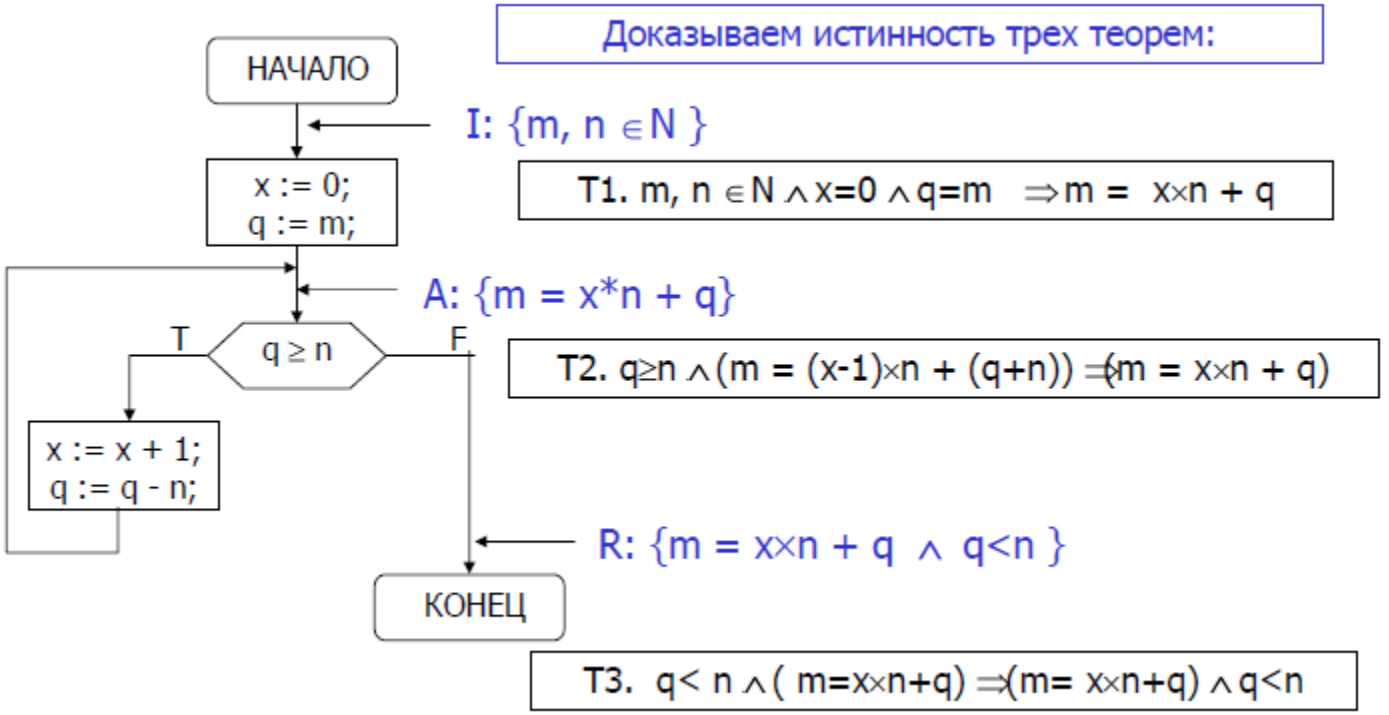
Но y всегда будет не равно n, потому что число n – дробное, а y - целое

Пример доказательства частичной корректности



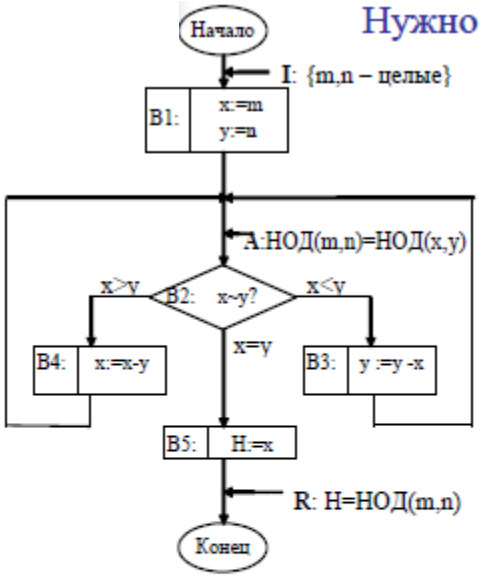
Пример: программа целочисленного деления

Программа находит частное x и остаток q от деления m на n



- Программа частично корректна. При n=0 она не завершается

Доказательство частичной корректности программы нахождения НОД



Нужно доказать четыре утверждения (теоремы):

T1. $sp(B1, \mathbf{I}) \Rightarrow \mathbf{A}$

T2. $sp(B3, sp(B2_{x<y}, A)) \Rightarrow \mathbf{A}$

T3. $sp(B4, sp(B2_{x>y}, A)) \Rightarrow \mathbf{A}$

T4. $sp(B5, sp(B2_{x=y}, A)) \Rightarrow \mathbf{R}$

или:

T1. m,n – целые $\wedge x=m \wedge y=n \Rightarrow$ НОД(m,n)=НОД(x,y)

T2. $x<y+x \wedge$ НОД(m,n)=НОД(x,y+x) $\Rightarrow$ НОД(m,n)=НОД(x,y)

T3. $x+y>y \wedge$ НОД(m,n)=НОД(x+y,y) $\Rightarrow$ НОД(m,n)=НОД(x,y)

T4. $x=y \wedge$ НОД(m,n)=НОД(x,y) $\wedge$ H=x $\Rightarrow$ H=НОД(m,n)]

Программа снабжается кроме **I** и **R** ***только одним*** доп. утверждением: ***инвариантом цикла*** A

Все четыре теоремы доказываются из свойств НОД

Программа ***частично корректна:*** при отрицательном m либо n программа входит в бесконечный цикл

Формулировка инварианта требует проникновения в суть алгоритма

Программирование по контракту (Design by Contract)

Автор этой идеи и языка программирования Eiffel - Бертран Мейер. Этот принцип реализован компанией Microsoft на платформе .net

В большинстве ОО языков семантические свойства классов определяются лишь в неформальных комментариях. Каждый класс содержит список операций, но информация о том, в каких условиях можно вызывать метод, какие параметры являются обязательными, а какие нет, и что следует ожидать вызывающему процессу в результате выполнения, не является формальной

"*Проектирование по контракту*" – формальное определение отношения между классами и их клиентами (то есть кодом, создающим экземпляры этих классов, и взаимодействующим с ними) *с помощью пред- и постусловий*, которое ФОРМАЛЬНО устанавливает права и обязанности сторон (***Assume-Guarantee*** подход) http://www.rsdn.ru/article/design/Code_Contracts.xml

Метод S.f( ) с утверждениями **require** *pre* и **ensure** *post* говорит:
«*Если вы обещаете вызвать метод f( ) класса S в состоянии, удовлетворяющем pre, то я гарантирую, что заключительное состояние удовлетворит post*».
Фактически, это гарантия истинности утверждения {pre} S.f( ) *{post}*

Пример программирования по контракту

```
//Stack.cs
 public void Push(T,t) {
        Contract.Requires ( t != null );
        Contract.Ensures(Count = = Contract.OldValue(Count)+1,"One more item");
   // Реализация метода
 }
```

Предусловие связывает вызывающий код: определяются условия, при которых вызов метода клиентом легитимен (например, x > 0 для метода Sqrt или Count != 0 для функции Pop класса Stack). При этом обязательства клиента приносят выгоду классу-поставщику, поскольку классу-поставщику, выполняющему метод, не нужно заботиться о том, что делать при нарушении предусловия: возвращать значение по умолчанию или код ошибки, генерировать исключение, сохранять информацию об ошибке в поток ввода-вывода или прерывать выполнение программы

Постусловие связывает класс: определяются условия, которые должны быть выполнены по завершению методов (класс Stack должен обеспечить увеличение количество элементов на 1 после выполнения метода Push). Здесь выгода клиента (результат выполнения метода) оборачивается обязательствами поставщика (он уже не может не выполнить свои обязательства, коль они прописаны в контракте).



СПАСИБО
ЗА ВНИМАНИЕ